Algorithm 1: RandOpt (PyTorch-style). **N**: population size, **K**: ensemble size. **sigmas**: noise scales, **theta**: model params

```
# Training: Select top-K seeds based on D_train performance
seeds = [sample_seed() for _ in range(N)]
## assign sigma to each seed
sigmas_per_seed = [sigmas[i // (N // len(sigmas))]
                   for i in range(N)]

## evaluate all perturbed models
scores = [evaluate(theta + sigmas_per_seed[i] * eps(seed[i]), D_train)
          for i in range(N)]
top_indices = topk(scores, K).indices

# Inference: Ensemble predictions on test input x
answers = [generate(theta + sigmas_per_seed[i] * eps(seed[i]), x)
           for i in top_indices]
prediction = majority_vote(answers)
```

Inference: Ensembling of Predictions. For a test input x, we generate predictions using only the selected set $\mathcal{I}_{\text{top}}$. The final output $\hat{y}$ is obtained by aggregating the individual predictions via majority voting:

$$\hat{y} = \text{mode}\left(\left\{\arg\max_{y} f_{\boldsymbol{\theta}_i}(y|x) \mid i \in \mathcal{I}_{\text{top}}\right\}\right) \tag{5}$$

RandOpt differs from standard practice in several ways. First, it does not involve gradient steps, and in fact does not involve sequential updates at all – it is entirely parallel. Second, it finds a *set* of solutions, which can be ensembled, rather than a *single* setting of the weights. This latter property has also been explored in the literature on evolutionary methods and quality-diversity algorithms, which maintain a population of promising solutions rather than collapsing on a single parameter vector (e.g., Mouret & Clune (2015); Jaderberg et al. (2017); Huang et al. (2017)).

5 How Does RandOpt Compare to Standard Methods for Post-Training?

We test RandOpt on post-training of LLMs and VLMs, and find it to be effective across a range of settings, often outperforming standard baselines. Although these results are strong, the reader should keep in mind that on these benchmarks performance can be sensitive to minor stylistic and formatting changes. Section 8 analyzes where the performance gains are coming from and argues that they are partially from reasoning improvements and partially from formatting improvements; notably, this is also true for certain baselines.

5.1 RandOpt on Large Language Models

We evaluate several models (Qwen, Llama, OLMo3; 0.5B–8B) covering both base and instruct variants, across four domains: math (Countdown, GSM8K, MATH-500, OlyBench), code (MBPP), writing (ROCStories), and chemistry (USPTO). We compare RandOpt against Test-Time Majority Vote (TT-MV), PPO, GRPO, and ES. Full details on datasets, models, and baselines are in Appendices A–C.

Our main finding is that RandOpt (with $K = 50$) mostly matches or outperforms established methods across a range of model scales (0.5B to 8B) and task categories. Benchmark performance compared to baselines, all run with equal training flops, is shown in Figure 6. More comparisons are given in Appendix D.

Notably, this performance is achieved while RandOpt involves no sequential optimization steps, whereas the baselines are run for hundreds of steps (see Appendix Table E.3 for hyperparameter settings). This gives RandOpt a potentially large wall-clock advantage, provided it is run on a large enough cluster of parallel compute.

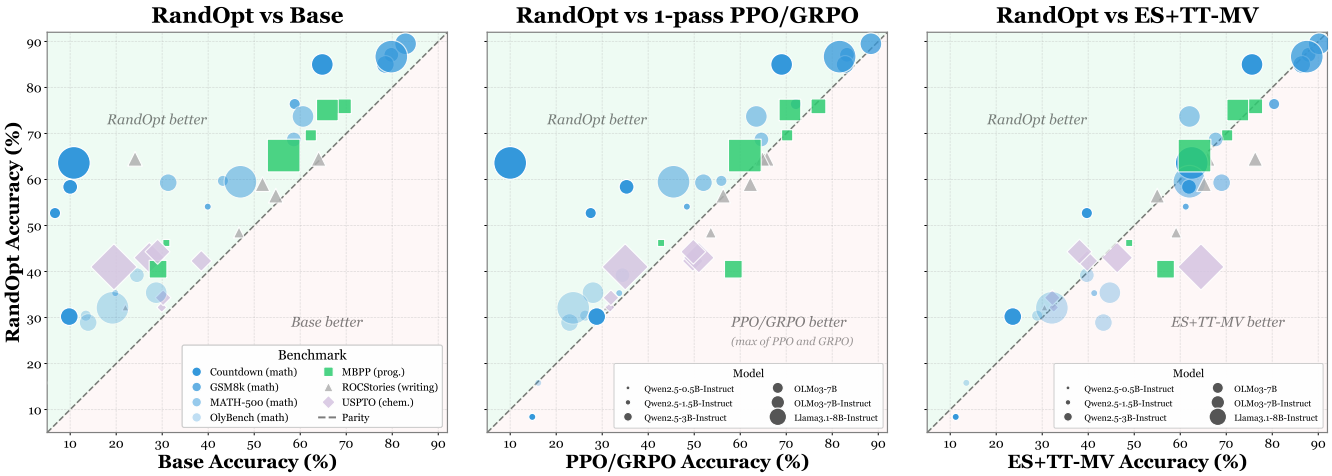

Figure 6: RandOpt vs. baselines on post-training LLMs. Marker size represents model scale (0.5-8B), shape indicates task family, and transparency distinguishes benchmarks within each task family. RandOpt matches or exceeds baselines in most settings. RandOpt is run with K=50 and ES+TT-MV also uses 50 test-time samples ensembled via majority vote. 1-pass PPO/GRPO use a single test-time sample; this setting disadvantages the baseline but reflects current standard usage of these models, in which there is no test-time ensembling. Full experimental results can be found in Appendix Table 4.

To demonstrate this, we deployed RandOpt on a 200 GH200 cluster and trained `Olmo-3-7B-Instruct` on Countdown. Using $N = 2000, K = 50$, this takes 3.2 minutes and achieves 70% accuracy.

These experiments also reveal that the ensembling phase is crucial to RandOpt's performance. RandOpt with K=1 is substantially less effective than RandOpt with K=50, as shown in Figure 1 (c) as well as Figure 11.

5.2 RandOpt on Vision-Language Models

We conduct experiments on Qwen2.5-VL-3B-Instruct, a 3B-parameter vision-language model (VLM). We evaluate on the Visual Reasoning in the Real World (GQA) dataset (Hudson & Manning, 2019), which contains questions requiring understanding of objects, attributes, and relations in images and is commonly used to benchmark visual reasoning ability. We perturb the language model while keeping the visual encoder frozen, and run RandOpt with $N = 5000$ and $K = 50$. This improves accuracy on GQA by 12.4% (Table 1).

Table 1: RandOpt improves Accuracy (%) on GQA.

Model	Method	GQA
Qwen2.5-VL-3B-Inst	Base	56.6
	RandOpt	69.0

5.3 Can Sandbagging Explain These Results?

Tice et al. (2025) argued that some models might be "sandbagged," where they are explicitly or implicitly trained to have low performance on certain tasks. Random weights perturbations might recover performance by breaking this effect. Can this explain our results? We think not. Most notably, RandOpt substantially improves the OLMo3-7B Base model (see Appendix Table 4). Since OLMo's training data and recipes are open-source, we can verify that this model is free of intentional sandbagging. We provide further arguments against sandbagging as an explanation in Appendix G. However, even if sandbagging is not the right explanation, this does not mean that there could not be similarly superficial fixes that underlie the performance gains; in Section 8 we look into this in more detail and find that indeed some, but not all, of the gains are due to simply fixing answer format.

5.4 Can Ensembling Also Benefit the Baselines?

Yes, ensembling (e.g., 50-pass TT-MV) consistently improves baseline methods across various model scales and benchmarks. For example, in Figure 1(c), it boosts the accuracy of PPO, GRPO, and ES to approximately 79% by

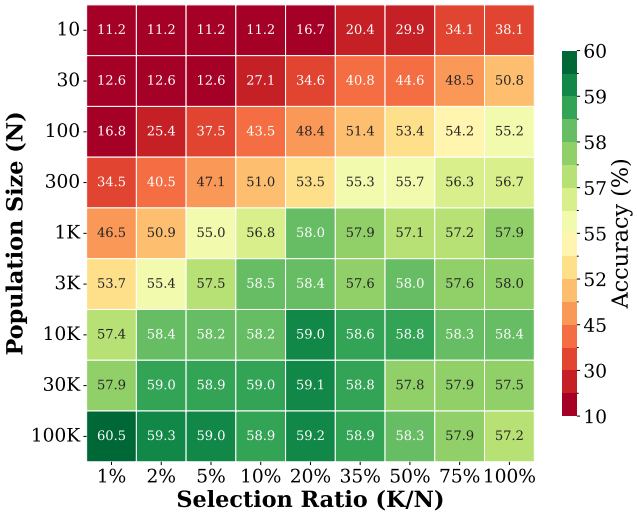

Figure 7: Heatmap of accuracy across population size N and selection ratio K/N. Accuracy scales with population size. Task: Countdown, Model: Qwen2.5-3B-Inst.

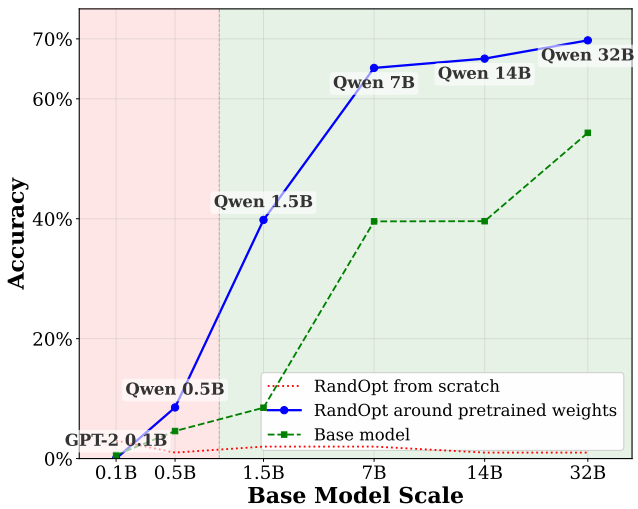

Figure 8: Relationship between model scale and RandOpt performance. Good pretrained representations are important for RandOpt start to work. Task: Countdown, N = 3k, K = 50.

step 500. Table 4 also suggests this. For example, ES + TT-MV increases the GSM8k accuracy of Qwen2.5-0.5B from 42.6% to 61.2%.

In fact, ensembling benefits these models regardless of the specific selection method used during training (e.g., random guessing, GRPO, or ES). Interestingly, as training progresses, the ensembled performance gap among these different baselines gradually shrinks (Figure 1(c)).

6 Scaling Properties of RandOpt

We investigate how RandOpt performance scales with respect to the search budget (population size N), the ensemble size (K), and the base model size.

Impact of Population Size and Selection Ratio. Figure 7 measures the interplay between search budget (N) and selection ratio (K/N) on the Countdown task, with Qwen2.5-3B-Inst (see Appendix Figure 10 for further slices of this data). Note that the precise shape of these trends may vary from task to task and model to model, and here we only show results for Countdown. Two observations:

(1) For sufficiently low selection ratio, performance improves monotonically with population size N.

(2) Trade-off between N and K/N: the optimal selection ratio decreases with increasing population size.

Note that the top-1 model's performance on the training set is, by construction, non-decreasing in N. This is consistent with observation #1; if a practitioner wants to avoid hyperparameter tuning, a reasonable strategy would therefore be to set K small and set N as large as their budget can afford.

The Emergence of Thickets at Scale. Figure 8 illustrates the relationship between base model scale and RandOpt's performance, showing that a strong base model is essential for RandOpt to start to work. The blue line corresponds to applying RandOpt to the base models, and the green dashed line is the performance of the base models. For very small models, such as GPT-2 0.1B, RandOpt fails to improve performance; for small models (e.g., Qwen 0.5B), RandOpt also offers small gains over the baseline. However, starting at around 1.5B parameters, RandOpt triggers a rapid increase in accuracy. After this point, the base model accuracy begins to catch up and the relative improvement of RandOpt shrinks as the models plateau. Using RandOpt on a model that has not been pretrained ("RandOpt from scratch," red dotted line) remains near zero across all scales. These results suggest that sufficient pretraining and sufficient model scale are essential for RandOpt to work; this matches our finding in Section 2 for the conditions under which thickets emerge.